

Raport z Testowania Aplikacji

Weryfikacja projektu **Terminal Arcade** została podzielona na testy automatyczne (logika i baza danych) oraz scenariusze testów manualnych (interakcja z konsolą).

1. Testy Automatyczne (Unit Tests)

`test`

wykonywane szybko i w dowolnym środowisku.

Jak uruchomić testy automatyczne:

```
# Kompilacja i uruchomienie
cmake -B build
cmake --build build
.\build\TerminalArcade.exe --test
```

Zakres testów automatycznych (w pliku `tests.cpp`):

1. **Logika gry Tic-Tac-Toe:** Testowanie metody `checkWin` dla różnych układów planszy 3x3. Upewnienie się, że zwycięskie kombinacje dla gracza `X` są poprawnie wykrywane, a brak wygranej gracza `O` zwraca wartość `false`.
2. **Logika klasy Database (Baza Danych):**

Przekierowanie zapisu na plik tymczasowy `test_scores_temp.txt` w celu ochrony realnych wyników.

Weryfikacja poprawności sortowania wyników w trybie **malejącym** (wyższe wyniki są lepsze, np. Snake, 2048).

Weryfikacja poprawności sortowania wyników w trybie **rosnącym** (niższe wyniki są lepsze, np. Saper, gdzie liczy się czas w sekundach).

Weryfikacja limitowania liczby zwracanych rekordów (odczyt top 2 z 3 zapisanych wyników).

Sprzątanie i usuwanie plików tymczasowych po zakończeniu testu.

1. **Logika gry Blackjack:** Weryfikacja metody `getHandValue` pod kątem sumowania punktów i dynamicznej zmiany wartości Asów (11 lub 1) w celu uniknięcia przekroczenia 21 punktów.
 2. **Logika gry 2048:** Testowanie funkcji `slideLine` w zakresie przesuwania kafelków w lewo, łączenia identycznych sąsiednich wartości oraz niełączenia różnych wartości.
 3. **Logika gry Minesweeper:** Sprawdzanie metody `checkWin` dla małej planszy 3x3 przy pełnym odkryciu, niepełnym odkryciu oraz grze w toku.
 4. **Logika gry CyberDefense:** Weryfikacja metod `hasTowerAt` i `getTowerIndex` w celu poprawnej lokalizacji wież na planszy, oraz `isEnemyInLane` pod kątem prawidłowego wykrywania wrogów na danej ścieżce i dystansie.
 5. **Logika gry Battleship:** Testowanie metody `canPlaceShip` (sprawdzenie poprawności granic planszy 10x10, wykrywanie nakładania się statków) oraz `checkWin` weryfikującej stan zatopienia floty.
-

2. Scenariusze Testów Manualnych

Ze względu na interaktywny charakter gier terminalowych, poprawność działania UI i sterowania jest weryfikowana manualnie zgodnie z poniższymi przypadkami testowymi:

Przypadek 1: Walidacja danych wejściowych w Minesweeper (Custom)

Kroki:

1. Uruchom aplikację.
2. Wybierz grę `Minesweeper` z menu głównego.
3. Wybierz konfigurację `Custom`.
4. Przy monicie o szerokość (`width`) wpisz ciąg znaków nieliterowych (np. `abc`) i zatwierdź.
5. Przy kolejnym monicie wpisz liczbę ujemną (np. `-5`).
6. Wpisz poprawną wartość (np. `10`).

Oczekiwany rezultat: Program powinien odrzucić niepoprawne typy i wartości spoza przedziału [5, 30], wyświetlić komunikat o błędzie i prosić o ponowne podanie liczby. Program pod żadnym pozorem nie może się zawiesić ani zapętlić.

Przypadek 2: Zabezpieczenie spójności bazy wyników (Tabela rekordów)

Kroki:

1. Zagraj w `Snake` (tryb Classic) i zdobądź punkty.
2. Po przegranej, przy monicie o wprowadzenie nazwy do tabeli wyników wpisz `A,B` (nazwa zawierająca przecinek).
3. Spróbuj wpisać nazwę ze spacją (np. `Jan Kowalski`).
4. Zatwierdź nazwę bez spacji i przecinków (np. `Gracz1`).

Oczekiwany rezultat: Program powinien odrzucić nazwy zawierające spacje lub przecinki, wyświetlając komunikat ostrzegawczy, a następnie zapisać poprawną nazwę `Gracz1` do pliku `scores.txt` i wyświetlić ją w Top 3 menu głównego.

Przypadek 3: Kod Konami (Easter Egg)

Kroki:

1. Będąc w menu głównym aplikacji Terminal Arcade, wpisz za pomocą klawiatury sekwencję: `W W S S A D A D B A`.

Oczekiwany rezultat: Aktywuje się **Tryb Tęczowy (Rainbow Mode)**. Kolory ramki i tytułu menu głównego powinny płynnie zmieniać barwę (losowo) przy każdym ruchu nawigacji. Ponowne wpisanie sekwencji powinno wyłączyć tryb.